

Artefakt 1: K40_BRM

- **Ausgangslage (LO-VC Input / Rohdaten):**

- Merkmalsname: K40_BRM (Charakter-Merkmal).
- Zugeordnete Werte: 0, 1 und 2.
- Wert 0 hat keine Vorbedingungen.
- Wert 1 hat die Vorbedingungen: V_TW0003 (K40_fg_af = '3') und V_001 (specified dummy).
- Wert 2 hat die Vorbedingung: V_TW0004 (K40_fg_af = '1').

- **Meine Analyse & Gedankengänge bei der Migration:**

- *Logik-Verknüpfung:* Wenn ein Wert mehrere Preconditions hat, müssen alle erfüllt sein (logisches AND).
- *Umgang mit "specified dummy":* Ich weiß aus Erfahrung mit den Kundendaten, dass der Ausdruck specified dummy immer dann verwendet wird, wenn ein Wert *nie* aufgerufen werden soll (da der Dummy nie klassifiziert wird). Das ist quasi historisch gewachsener "toter Code". Meine Entscheidung: Ich lasse Wert 1 komplett bei der Migration weg.
- *Struktur-Entscheidung:* Da Wert 0 keine Bedingung hat und Wert 1 gelöscht wird, verbleibt nur noch Wert 2 mit einer Bedingung. Da es nur ein einziger Wert ist, schreibe ich das als direkte Code-Zeile und schränke es nicht über eine extra Variantentabelle ein.
- *AVC-Syntax-Anpassung:* Ich war mir anfangs unsicher, wie der genaue AVC-Code aussehen soll. Nach Blick in die Dokumentation lasse ich den alten INFERENCES-Block komplett weg, da AVC ohnehin alles herleitet. Auch den umständlichen OBJECTS-Block spare ich mir und referenziere die Merkmale direkt über das Prefix \$self..

- **Mein erstellter AVC Ziel-Code:**

Code-Snippet

RESTRICTIONS:

```
$self.K40_BRM = '2' if $self.K40_fg_af = '1'.
```

Artefakt 2: K40_KO

- **Ausgangslage (LO-VC Input / Rohdaten):**

- Merkmalsname: K40_KO mit den Werten 01 bis 10 sowie 99.

- Viele Werte haben komplexe, teils redundante Vorbedingungen im JSON-Format hinterlegt.
- **Meine Analyse & Gedankengänge bei der Migration:**
 - *Bereinigung toter Code:* Die Werte 04, 06, 09 und 10 haben wieder die Vorbedingung specified dummy. Diese ignoriere ich bei der Ziel-Generierung komplett.
 - *Bereinigung Kommentare:* Bei Wert 99 sind drei der vier Zeilen der Vorbedingung mit einem Stern (*) auskommentiert. Es verbleibt für diesen Wert also eigentlich nur eine echte, aktive Vorbedingung.
 - *Mustererkennung & Tabellenbildung:* Mir fällt auf, dass sich die Werte 01, 02 und 03 exakt dieselbe Tabellen-Vorbedingung teilen. Das Gleiche gilt für die Werte 05 und 07. Wenn ich das alles als einzelne IF-Zuweisungen ausschreibe, wird der Code endlos lang. Ich entscheide mich, diese Cluster in zwei Variantentabellen zusammenzufassen.
 - *Syntax-Update:* Wert 08 hat eine sehr eigenständige Vorbedingung, in der ne (not equal) vorkommt. Die korrekte Syntax hierfür in der modernen AVC-Engine ist <>. Das muss ich beim Schreiben ersetzen.

- **Mein erstellter AVC Ziel-Code:**

Code-Snippet

RESTRICTIONS:

```
table K40_KO_01 (
    K40_fg_af = $self.K40_fg_af,
    K40_abl = $self.K40_abl,
    K40_KO = $self.K40_KO
),
table K40_KO_02 (
    K40_fg_af = $self.K40_fg_af,
    K40_ts = $self.K40_ts,
    K40_KO = $self.K40_KO
),
$self.K40_KO = '08' if ($self.K40_fg_af = '3' or $self.K40_fg_af = '4')
and $self.K40_brm = '0' and $self.K40_ts <> '1',
```

`$self.K40_KO = '99' if $self.K40_pl in ('d1','h1','h2').`

Artefakt 3: K40_PL

- **Ausgangslage (LO-VC Input / Rohdaten):**

- Merkmalsname: K40_PL (Char).
- Zugeordnete Werte: H2, H3, H5, D1, H1, H4, D2.
- Werte H2, H3 und H5 enthalten alle die Bedingung specified dummy. Bei H2 steht diese in Kombination mit einer weiteren Bedingung (K40_fst = '0').
- Werte D1 und H1 haben beide exakt dieselbe Bedingung: K40_fst = '0'.
- Wert H4 hat die Bedingung: Specified K40_FG_AF and K40_FG_AF = '4'.
- Wert D2 hat keine Vorbedingungen.

- **Meine Analyse & Gedankengänge bei der Migration:**

- *Umgang mit Dummy-Werten:* Auch hier taucht das Anti-Pattern specified dummy wieder auf. Die Werte H3 und H5 fliegen direkt raus. Interessant ist H2: Hier gibt es noch eine zweite Vorbedingung. Da Vorbedingungen bei Werten aber logisch mit AND verknüpft werden, zieht der Dummy den kompletten Wert mit runter. Ich lösche H2 also ebenfalls komplett aus der Ziel-Architektur.
- *Redundanz-Bereinigung:* Bei Wert H4 steht Specified K40_FG_AF and K40_FG_AF = '4'. Das ist logisch betrachtet "doppelt gemoppelt". Wenn ein Merkmal den Wert '4' hat, ist es automatisch auch spezifiziert. Ich entscheide mich, diesen alten Ballast abzuwerfen und den AVC-Code schlanker zu machen, indem ich das Specified einfach weglasse.
- *Mustererkennung & Tabellenbildung:* Die Werte D1 und H1 teilen sich wieder exakt dieselbe Bedingung (K40_fst = '0'). Um den Code elegant zu halten und die Engine nicht mit IF-Statements zu überladen, packe ich diese beiden Werte in eine kleine Variantentabelle.
- *Wert ohne Bedingung:* Da D2 keine Bedingung hat, muss ich dafür keinen Code generieren. Er ist einfach immer erlaubt.

- **Mein erstellter AVC Ziel-Code:**

Code-Snippet

RESTRICTIONS:

table K40_PL_01 (

```
K40_fst = $self.K40_fst,  
  
K40_PL = $self.K40_PL  
  
)  
  
$self.K40_PL = 'H4' if $self.K40_FG_AF = '4'.
```

- **Meine Tabellen hierzu:**

- **Tabelle 1 (K40_PL_01):**
 - Spalten: K40_fst, K40_PL
 - Zeilen: 0, D1; 0, H1

Artefakt 4: K40_SCHP

- **Ausgangslage (LO-VC Input / Rohdaten):**

- Merkmalsname: K40_SCHP (Char) mit den Werten 01 bis 10.
- Besonderheiten bei den Vorbedingungen:
 - Wert 02: Enthält einen großen Block an auskommentiertem Freitext (z. B. *Merkmalswert ausgeblendet) und nur eine aktive Zeile (not specified K40_FG_AF).
 - Werte 01 und 10: Haben exakt dieselbe Bedingung (K40_ko_af = '1' and K40_ps ne 'pw04' and K40_ps ne 'ps03').
 - Werte 07 und 08: Haben exakt dieselbe Bedingung (K40_ko_af = '2' and K40_fg_af in ('3','4') and K40_ko ne '07').
 - Werte 03 und 09: Haben exakt dieselbe Bedingung (K40_ko = '08' and K40_abl = '0').
 - Wert 05: Ähneln 03/09, prüft aber auf K40_ko = '09'.

- **Meine Analyse & Gedankengänge bei der Migration:**

- *Code-Bereinigung:* Bei Wert 02 hat früher offensichtlich jemand die Dokumentation direkt in den Code geschrieben und alte Logik auskommentiert (*). Das muss rigoros gelöscht werden, es zählt nur die aktive Zeile.
- *Syntax-Update:* Der veraltete Operator ne taucht in mehreren Bedingungen auf. Dieser muss zwingend in <> übersetzt werden.
- *Struktur-Entscheidung (Redundanzen bündeln):* Es gibt hier massive Dopplungen. Da einige Bedingungen Ungleichungen (ne bzw. <>) enthalten, eignen sich diese nicht optimal für klassische

Variantentabellen (da Tabellen meist auf reine Gleichheit prüfen). Daher entscheide ich mich hier für einen anderen Weg der Bündelung: Ich schreibe die Restriktionen nicht einzeln pro Wert, sondern fasse die Werte im IF-Statement über Listen-Abfragen (in) zusammen. Das hält den Code ebenfalls sehr kompakt und ist für die AVC-Engine schnell auswertbar.

- **Mein erstellter AVC Ziel-Code:**

Code-Snippet

RESTRICTIONS:

\$self.K40_SCHP = '02' if not specified \$self.K40_FG_AF,

\$self.K40_SCHP = '06' if \$self.K40_brm = '0' and \$self.K40_abl = '0',

\$self.K40_SCHP = '04' if \$self.K40_ko = '09' and not specified \$self.K40_ps,

\$self.K40_SCHP in ('03', '09') if \$self.K40_ko = '08' and \$self.K40_abl = '0',

\$self.K40_SCHP = '05' if \$self.K40_ko = '09' and \$self.K40_abl = '0',

\$self.K40_SCHP in ('01', '10') if \$self.K40_ko_af = '1' and \$self.K40_ps <> 'pw04' and \$self.K40_ps <> 'ps03',

\$self.K40_SCHP in ('07', '08') if \$self.K40_ko_af = '2' and \$self.K40_fg_af in ('3','4') and \$self.K40_ko <> '07'.

Artefakt 5: K40_GK_FB

- **Ausgangslage (LO-VC Input / Rohdaten):**

- Merkmalsname: K40_GK_FB (Char) mit insgesamt 10 Werten (Farb- bzw. Strukturcodes wie 1018, 3000, 2004 etc.).
- Besonderheit: Die ersten 7 Werte (1018, 3000, 5002, 5003, 6024, 2002, 7016) haben alle exakt dieselbe Vorbedingung hinterlegt: V_TW0228 mit der Syntax K40_grh_af ne '3'.
- Die verbleibenden 3 Werte (2004, 7032, 7035) besitzen überhaupt keine Vorbedingungen.

- **Meine Analyse & Gedankengänge bei der Migration:**

- *Mustererkennung & Redundanz:* Das ist ein klassisches Beispiel für ineffiziente Datenpflege im Altsystem. Anstatt die Bedingung einmal zentral abzuprüfen, wurde sie an 7 verschiedene Einzelwerte gehängt.
- *Struktur-Entscheidung (Architektur-Wahl):* Wenn ich das 1:1 in AVC übersetze, müsste ich ein IF-Statement mit einer langen Liste (IN ('1018', '3000', ...)) schreiben. Da es hier aber ausschließlich um eine einfache Bedingung mit einer Ungleichung (ne) geht und die anderen drei Werte ohnehin immer gelten, entscheide ich mich für eine elegantere Logik-Umkehr. Anstatt positiv aufzulisten, welche 7 Werte bei <> '3' erlaubt sind, schreibe ich ein Constraint, das definiert, was passiert, wenn der Wert *doch* '3' ist (dann sind eben nur noch die drei freien Werte erlaubt).
- *Syntax-Update:* Der Operator ne wird in AVC durch <> ersetzt.

- **Mein erstellter AVC Ziel-Code:**

Code-Snippet

RESTRICTIONS:

`$self.K40_GK_FB in ('2004', '7032', '7035') if $self.K40_grh_af = '3'.`